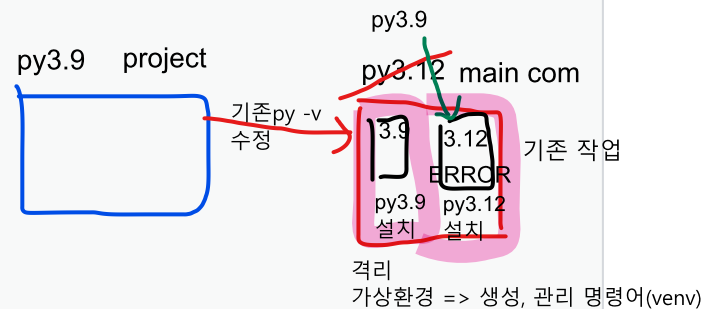


Windows 11에서 Python venv 사용법

가상 환경(virtual environment) 생성부터 관리까지 상세 가이드

목차

1. venv란 무엇인가
2. 사전 준비: Python 설치 확인
3. 가상 환경 생성하기
4. 가상 환경 활성화 / 비활성화
5. PowerShell 실행 정책 문제 해결
6. 패키지 설치와 관리 (pip)
7. requirements.txt로 환경 재현하기
8. 가상 환경 삭제
9. VS Code에서 venv 사용
10. 자주 발생하는 문제 (FAQ)
11. 명령어 요약 치트시트



1. venv란 무엇인가

venv : python-org에서 만들
conda
uv
...

venv 는 Python 프로젝트마다 독립된 패키지 환경을 만들어 주는 표준 모듈입니다. 프로젝트별로 서로 다른 라이브러리 버전을 충돌 없이 사용할 수 있습니다.

- 격리: 프로젝트 A는 Django 4.2, 프로젝트 B는 Django 5.1을 쓰더라도 서로 간섭하지 않습니다.
- 시스템 보호: 전역(system) Python을 오염시키지 않아, 실수로 시스템 패키지를 망가뜨릴 위험이 없습니다.
- 재현성: 설치한 패키지 목록을 파일로 남겨 다른 PC에서 동일한 환경을 그대로 복원할 수 있습니다.
- 기본 내장: Python 3.3 이상에 기본 포함되어 있어 별도 설치가 필요 없습니다.

2. 사전 준비: Python 설치 확인

먼저 Python이 설치되어 있는지 확인합니다. PowerShell 또는 명령 프롬프트(cmd) 를 열고 다음을 입력하세요.

```
python --version
```

Python 3.12.x 처럼 버전이 출력되면 정상입니다. 만약 Microsoft Store 페이지가 뜨거나 인식되지 않으면 아래 방법으로 설치합니다.

설치 방법 A — 공식 설치 프로그램

python.org/downloads 에서 Windows용 설치 파일을 받아 실행합니다. 설치 첫 화면에서 "Add python.exe to PATH" 체크박스를 반드시 켜야 합니다.

설치 방법 B — winget (Windows 11 기본 제공)

```
winget install Python.Python.3.12
```

참고: Windows에서는 `python` 대신 `py` (Python Launcher)를 쓸 수도 있습니다. 여러 버전이 설치된 경우 `py -3.12` 처럼 버전을 지정할 수 있어 편리합니다.

3. 가상 환경 생성하기

프로젝트 폴더로 이동한 뒤 가상 환경을 생성합니다. 아래 예시는 `myproject` 폴더 안에 `venv` 라는 이름의 환경을 만듭니다.

```
cd C:\Users\사용자명\myproject
python -m venv venv
```

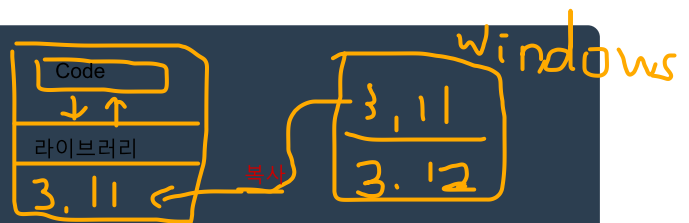
가상환경 생성 명령 이름(현재 디렉토리 아래 venv 디렉토리 생성)

맨 뒤의 `venv` 는 환경 폴더의 이름입니다. 관례적으로 `venv` 또는 `.venv` 를 많이 사용합니다.

```
# .venv 라는 숨김 폴더로 만들고 싶을 때
python -m venv .venv

# 특정 Python 버전으로 생성 (Launcher 사용)
py -3.11 -m venv venv
```

특정 버전 명시 가능



windows에서 설치한 버전이어야 가상환경에 복사 가능

생성이 완료되면 폴더 안에 다음과 같은 구조가 만들어집니다.

```
venv\
├── Scripts\      ← 가상환경 활성화 인터프리터      패키지 관리자 => 프로그램 추가/제거, 라이브러리 추가/삭제
├── Lib\          ← 실행 파일 (activate, python.exe, pip.exe 등)
├── pyvenv.cfg    ← 설치된 패키지가 저장되는 곳      가상 환경 안에 library 별도 관리
└── Include      ← 환경 설정 파일
                  ←- 내장
```

팁: Git을 사용한다면 `.gitignore` 파일에 `venv/` 또는 `.venv/` 를 추가하세요. 가상 환경 폴더는 저장소에 올리 지 않고, 대신 `requirements.txt` 만 공유하는 것이 표준입니다.

4. 가상 환경 활성화 / 비활성화

가상 환경은 사용하기 전에 반드시 **활성화(activate)** 해야 합니다. Windows에서는 사용하는 셸(shell)에 따라 명령이 다릅니다.

PowerShell (Windows 11 기본 터미널)

```
.\venv\Scripts\Activate.ps1
```

명령 프롬프트 (cmd)

```
venv\Scripts\activate.bat
```

Git Bash

```
source venv/Scripts/activate
```

활성화되면 프롬프트 맨 앞에 **(venv)** 표시가 나타납니다.

```
(venv) PS C:\Users\사용자명\myproject>
```

작업이 끝나면 다음 명령으로 **비활성화**합니다. 셸 종류와 무관하게 동일합니다.

```
deactivate
```

활성화가 왜 필요한가요? 활성화하면 **python** 과 **pip** 명령이 시스템이 아니라 가상 환경 안의 실행 파일을 가리키도록 PATH가 임시로 바뀝니다. 그래서 이후 설치하는 패키지가 모두 그 환경에만 담깁니다.

5. PowerShell 실행 정책 문제 해결

PowerShell에서 **Activate.ps1** 을 실행할 때 아래와 같은 오류가 자주 발생합니다.

오류 예시: "이 시스템에서 스크립트를 실행할 수 없으므로 ... Activate.ps1 파일을 로드할 수 없습니다."
(**UnauthorizedAccess** / execution policy 오류)

이는 Windows의 스크립트 실행 정책이 기본적으로 제한되어 있기 때문입니다. 다음 명령으로 **현재 사용자 한정**으로 정책을 완화하면 해결됩니다.

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser
```

실행 후 **Y** 를 입력해 확정합니다. 이후 다시 **Activate.ps1** 을 실행하면 정상 동작합니다.

- **RemoteSigned** : 로컬에서 만든 스크립트는 허용, 인터넷에서 받은 스크립트는 서명 필요. 보안과 편의의 균형이 좋아 권장됩니다.
- **-Scope CurrentUser** : 관리자 권한 없이 현재 사용자 계정에만 적용됩니다.

일회성 우회: 정책을 바꾸기 싫다면 그때그때 아래처럼 실행할 수도 있습니다.

```
powershell -ExecutionPolicy Bypass -File .\venv\Scripts\Activate.ps1
```

6. 패키지 설치와 관리 (pip)

가상 환경을 **활성화한 상태**에서 **pip** 로 패키지를 설치합니다.

```
# 패키지 설치
pip install requests

# 특정 버전 설치
pip install django==5.1

# 여러 개 한 번에 설치
pip install numpy pandas matplotlib

# 설치된 패키지 목록 확인
pip list

# 특정 패키지 상세 정보
pip show requests

# 패키지 업그레이드
pip install --upgrade requests

# 패키지 삭제
pip uninstall requests
```

pip 최신화: 새 환경에서는 먼저 pip를 업그레이드하는 것이 좋습니다.

```
python -m pip install --upgrade pip
```

7. requirements.txt로 환경 재현하기

설치한 패키지 목록을 파일로 저장해 두면, 다른 PC나 협업자가 동일한 환경을 그대로 만들 수 있습니다.

현재 환경 내보내기

```
pip freeze > requirements.txt
```

생성된 `requirements.txt`에는 다음처럼 패키지와 정확한 버전이 기록됩니다.

```
requests==2.32.3  
numpy==2.1.0  
pandas==2.2.2
```

다른 PC에서 환경 복원하기

```
# 1) 새 가상 환경 생성 후 활성화  
python -m venv venv  
.\venv\Scripts\Activate.ps1  
  
# 2) 목록대로 한 번에 설치  
pip install -r requirements.txt
```

8. 가상 환경 삭제

venv는 특별한 제거 절차가 없습니다. **폴더를 그냥 삭제**하면 됩니다. 먼저 비활성화한 뒤 폴더를 지웁니다.

```
deactivate  
  
# PowerShell에서 폴더 삭제  
Remove-Item -Recurse -Force venv  
  
# 또는 명령 프롬프트에서  
rmdir /s /q venv
```

탐색기에서 `venv` 폴더를 마우스로 삭제해도 무방합니다.

9. VS Code에서 venv 사용

Visual Studio Code는 가상 환경을 자동으로 인식합니다.

1. VS Code에서 프로젝트 폴더를 엽니다.
2. `Ctrl + Shift + P` → **"Python: Select Interpreter"** 입력.
3. 목록에서 `.\venv\Scripts\python.exe` (경로에 `venv` 포함)를 선택합니다.
4. 이후 새 터미널(`Ctrl + ``)을 열면 자동으로 venv가 활성화됩니다.

참고: 프로젝트 폴더 안에 `.venv` 이름으로 만들어 두면 VS Code가 더 잘 자동 감지합니다.

10. 자주 발생하는 문제 (FAQ)

증상	원인 / 해결
Activate.ps1 실행 시 스크립트 오류	실행 정책 문제. 5번 항목의 <code>Set-ExecutionPolicy RemoteSigned -Scope CurrentUser</code> 실행.
<code>python</code> 명령을 찾을 수 없음	PATH 미등록. 설치 시 "Add to PATH"를 체크하거나 <code>py</code> 사용.
활성화했는데 pip가 전역에 설치됨	프롬프트에 <code>(venv)</code> 가 있는지 확인. <code>where python</code> 으로 실제 경로 점검.
Store 버전 Python이 자주 실행됨	설정 → 앱 → 앱 실행 별칭에서 Python 앱 별칭을 끄기.
다른 폴더로 venv를 옮겼더니 동작 안 함	venv는 절대경로에 묶여 이동 불가. 삭제 후 재생성.

현재 어떤 python이 활성화인지 확인:

`where python` (cmd) 또는 `Get-Command python` (PowerShell)로 실제 실행 파일 경로를 확인할 수 있습니다. 경로에 `venv\Scripts` 가 보이면 정상입니다.

11. 명령어 요약 치트시트

작업	명령어
Python 버전 확인	<code>python --version</code>
가상 환경 생성	<code>python -m venv venv</code>
활성화 (PowerShell)	<code>.\venv\Scripts\Activate.ps1</code>
활성화 (cmd)	<code>venv\Scripts\activate.bat</code>
비활성화	<code>deactivate</code>
실행 정책 완화	<code>Set-ExecutionPolicy RemoteSigned -Scope CurrentUser</code>
패키지 설치	<code>pip install 패키지명</code>
목록 저장	<code>pip freeze > requirements.txt</code>
목록 설치	<code>pip install -r requirements.txt</code>
환경 삭제	<code>Remove-Item -Recurse -Force venv</code>